

MATH1005 Notes

Antriksh Dhand

29/11/2021

Contents

1	General R Things	3
1.1	Reading in data	3
1.2	Outputting dimension of data	3
1.3	Classification of variables (datatypes)	3
1.4	Difference between \$ and [] access operators	4
2	Data Summaries	5
2.1	Qualitative data	5
2.1.1	Bar plot (frequency of 1 categorical variable)	5
2.1.2	Double bar plot (frequency of 2 categorical variables) [Stacked or side-by-side]	7
2.2	Adjusting axis labels on a chart	13
2.2.1	las	13
2.2.2	cex	13
2.2.3	Adding lines and points to a graph	15
2.3	Quantitative data	16
2.3.1	Histogram (frequency of 1 quantitative variable)	16
2.3.2	Multiple histograms (frequency of 1 quantitative variable split by a categorical variable)	18
2.3.3	Single boxplot (numerical summary of a quantitative variable)	19
2.3.4	Multiple boxplots (numerical summary of a quantitative variable split up by a qualitative variable)	20
2.3.5	Scatter plots (compare two quantitative variables)	21
2.3.6	Heatmaps	21
3	Numerical summaries: Centre	23
3.1	Sample mean	23
3.2	Sample median	23
4	Numerical summaries: Spread	24
4.1	Sample SD	24
4.2	Population SD	24
4.3	Z-Score	25
4.4	IQR	25
4.5	Lower and upper thresholds	25
4.6	Coefficient of variation	25
5	General Normal Curve	25
6	Scatter plots and correlation	25
6.1	Correlation coefficient	25
6.2	Regression lines	27
6.2.1	SD line (bad descriptor)	27

6.2.2	The regression line	27
6.3	Residuals	29
6.3.1	Residual plot	29
7	Chance	29
7.1	Simulating chance	29

1 General R Things

1.1 Reading in data

```
data <- read.csv("WEEK 2/2020fatalities.csv")
```

1.2 Outputting dimension of data

```
dim(data)
```

```
## [1] 51001    23
```

1.3 Classification of variables (datatypes)

```
str(data)
```

```
## 'data.frame':    51001 obs. of  23 variables:
## $ Crash.ID      : int  20203010 20201006 20203001 20202004 20203002 20204006 20202011 ...
## $ State         : chr   "Qld" "NSW" "Qld" "Vic" ...
## $ Month         : int   1 1 1 1 1 1 1 1 1 1 ...
## $ Year          : int  2020 2020 2020 2020 2020 2020 2020 2020 2020 2020 ...
## $ Dayweek       : chr   "Sunday" "Sunday" "Friday" "Sunday" ...
## $ Time          : chr   "0:00" "4:25" "23:00" "23:25" ...
## $ Crash.Type    : chr   "Single" "Single" "Pedestrian" "Pedestrian" ...
## $ Bus.Involvement : chr   "No" "No" "No" "-9" ...
## $ Heavy.Rigid.Truck.Involvement: chr   "No" "No" "No" "-9" ...
## $ Articulated.Truck.Involvement: chr   "No" "No" "No" "-9" ...
## $ Speed.Limit   : chr   "100" "110" "10" "-9" ...
## $ Road.User     : chr   "Passenger" "Passenger" "Pedestrian" "Pedestrian" ...
## $ Gender        : chr   "Male" "Male" "Male" "Male" ...
## $ Age           : int   16 19 17 39 27 27 25 39 35 19 ...
## $ National.Remoteness.Areas : chr   "Inner Regional Australia" "Major Cities of Australia" "Major ...
## $ SA4.Name.2016 : chr   "Darling Downs - Maranoa" "Sydney - Outer South West" "Brisban ...
## $ National.LGA.Name.2017 : chr   "Toowoomba (R)" "Wollondilly" "Brisbane (C)" "Latrobe (C) (Vi ...
## $ National.Road.Type : chr   "Sub-arterial Road" "National or State Highway" "National or ...
## $ Christmas.Period : chr   "No" "No" "No" "No" ...
## $ Easter.Period   : chr   "No" "No" "No" "No" ...
## $ Age.Group       : chr   "0_to_16" "17_to_25" "17_to_25" "26_to_39" ...
## $ Day.of.week     : chr   "Weekend" "Weekend" "Weekend" "Weekend" ...
## $ Time.of.day     : chr   "Night" "Night" "Night" "Night" ...
```

```
# Changing the datatype of specific variables
```

```
data$Crash.ID <- as.factor(data$Crash.ID)
```

```
# note that you cannot use the levels argument here
```

```
# as Month was originally encoded from 1 - 12.
```

```
data$Month <- as.factor(data$Month)
```

```
data$Dayweek <- factor(data$Dayweek,
                       levels = c("Monday", "Tuesday", "Wednesday",
                                   "Thursday", "Friday", "Saturday",
                                   "Sunday"))
```

```
# the factor function encodes a vector as a factor (category)
```

```
# with order based on the levels argument
```

```
# converting to `logical` data format using the batman package
```

```
library(batman)
data$Christmas.Period = to_logical(data$Christmas.Period)
data$Easter.Period = to_logical(data$Easter.Period)
```

```
str(data)
```

```
## 'data.frame':    51001 obs. of  23 variables:
## $ Crash.ID      : Factor w/ 45876 levels "19891001","19891002",...: 45852 45806 45843 ...
## $ State         : chr  "Qld" "NSW" "Qld" "Vic" ...
## $ Month         : Factor w/ 12 levels "1","2","3","4",...: 1 1 1 1 1 1 1 1 1 1 ...
## $ Year          : int  2020 2020 2020 2020 2020 2020 2020 2020 2020 2020 ...
## $ Dayweek       : Factor w/ 7 levels "Monday","Tuesday",...: 7 7 5 7 5 5 3 2 3 1 ...
## $ Time          : chr  "0:00" "4:25" "23:00" "23:25" ...
## $ Crash.Type    : chr  "Single" "Single" "Pedestrian" "Pedestrian" ...
## $ Bus.Involvement : chr  "No" "No" "No" "-9" ...
## $ Heavy.Rigid.Truck.Involvement: chr  "No" "No" "No" "-9" ...
## $ Articulated.Truck.Involvement: chr  "No" "No" "No" "-9" ...
## $ Speed.Limit   : chr  "100" "110" "10" "-9" ...
## $ Road.User     : chr  "Passenger" "Passenger" "Pedestrian" "Pedestrian" ...
## $ Gender        : chr  "Male" "Male" "Male" "Male" ...
## $ Age           : int  16 19 17 39 27 27 25 39 35 19 ...
## $ National.Remoteness.Areas : chr  "Inner Regional Australia" "Major Cities of Australia" "Major ...
## $ SA4.Name.2016 : chr  "Darling Downs - Maranoa" "Sydney - Outer South West" "Brisban ...
## $ National.LGA.Name.2017 : chr  "Toowoomba (R)" "Wollondilly" "Brisbane (C)" "Latrobe (C) (Vi ...
## $ National.Road.Type : chr  "Sub-arterial Road" "National or State Highway" "National or S ...
## $ Christmas.Period : logi  FALSE FALSE FALSE FALSE TRUE FALSE ...
## $ Easter.Period    : logi  FALSE FALSE FALSE FALSE FALSE FALSE ...
## $ Age.Group        : chr  "0_to_16" "17_to_25" "17_to_25" "26_to_39" ...
## $ Day.of.week      : chr  "Weekend" "Weekend" "Weekend" "Weekend" ...
## $ Time.of.day      : chr  "Night" "Night" "Night" "Night" ...
```

1.4 Difference between \$ and [] access operators

```
year <- data$Year
year1 <- data["Year"]
head(year)
```

```
## [1] 2020 2020 2020 2020 2020 2020
```

```
head(year1)
```

```
##   Year
## 1 2020
## 2 2020
## 3 2020
## 4 2020
## 5 2020
## 6 2020
```

Using \$ allows you to access individual entries, whereas [] returns a vector/dataframe type object.

2 Data Summaries

2.1 Qualitative data

Often times when working with qualitative data summaries, we need to first produce a frequency table. This can be done using the `table()` function.

```
# outputs a frequency table
```

```
table(year)
```

```
## year
```

```
## 1989 1990 1991 1992 1993 1994 1995 1996 1997 1998 1999 2000 2001 2002 2003 2004
```

```
## 2800 2331 2113 1974 1953 1928 2017 1767 1755 1764 1817 1737 1715 1621 1583
```

```
## 2005 2006 2007 2008 2009 2010 2011 2012 2013 2014 2015 2016 2017 2018 2019 2020
```

```
## 1627 1598 1603 1437 1491 1353 1277 1300 1187 1151 1204 1292 1221 1135 1194 86
```

```
table(year)[1] # accesses the first element of the frequency table
```

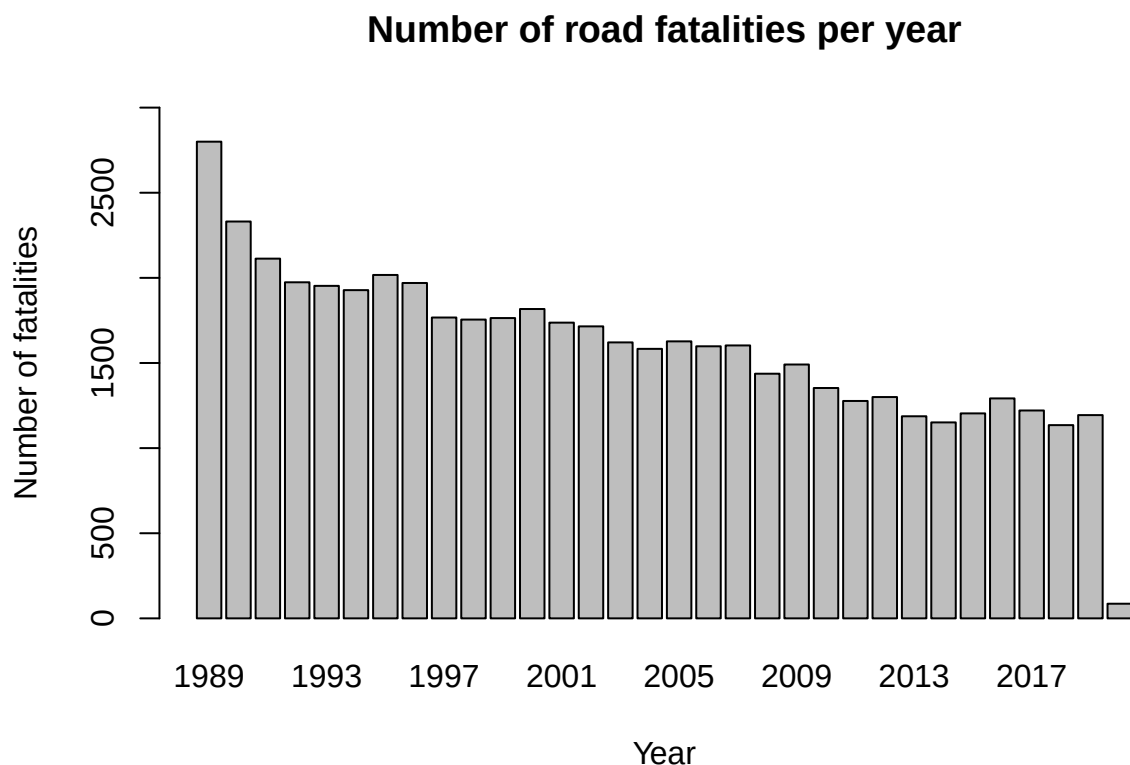
```
## 1989
```

```
## 2800
```

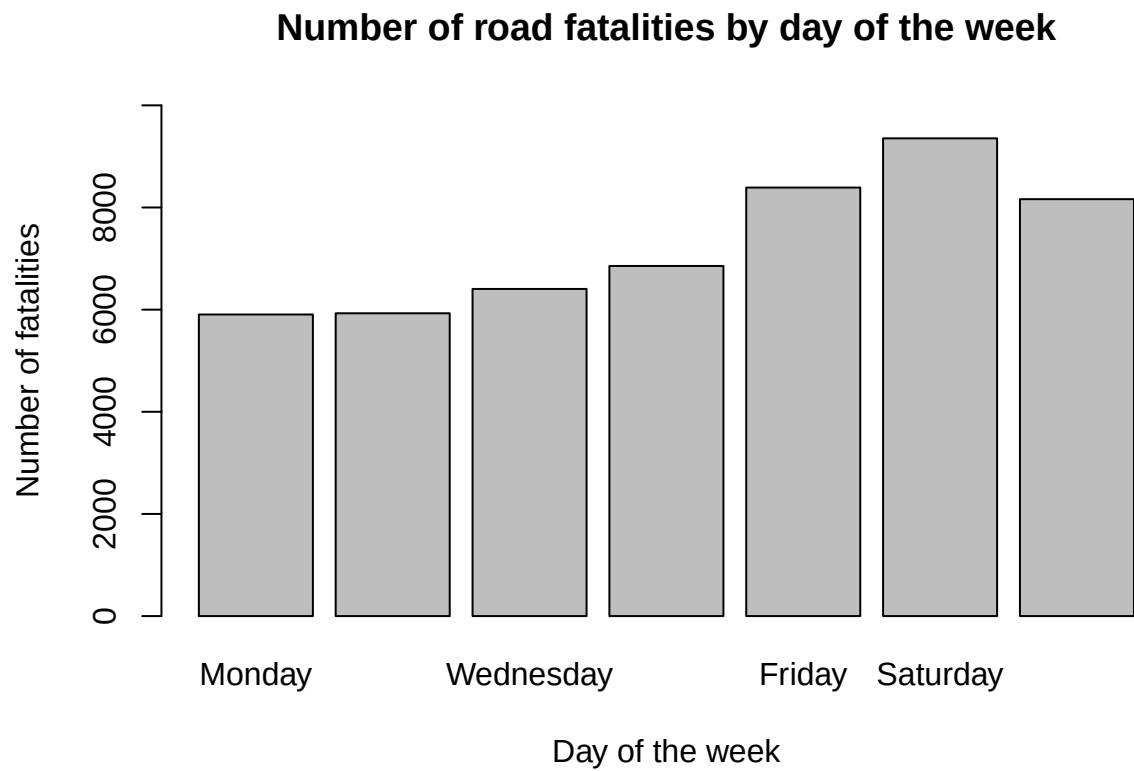
2.1.1 Bar plot (frequency of 1 categorical variable)

To produce a simple bar plot which counts the frequency of each category, use the `barplot` function which takes in a frequency table as its input.

```
barplot(table(year),  
        main = "Number of road fatalities per year",  
        xlab = "Year",  
        ylab = "Number of fatalities",  
        ylim = c(0, 3000))
```



```
dayweek <- data$Dayweek
barplot(table(dayweek),
        main = "Number of road fatalities by day of the week",
        xlab = "Day of the week",
        ylab = "Number of fatalities",
        ylim = c(0, 10000))
```

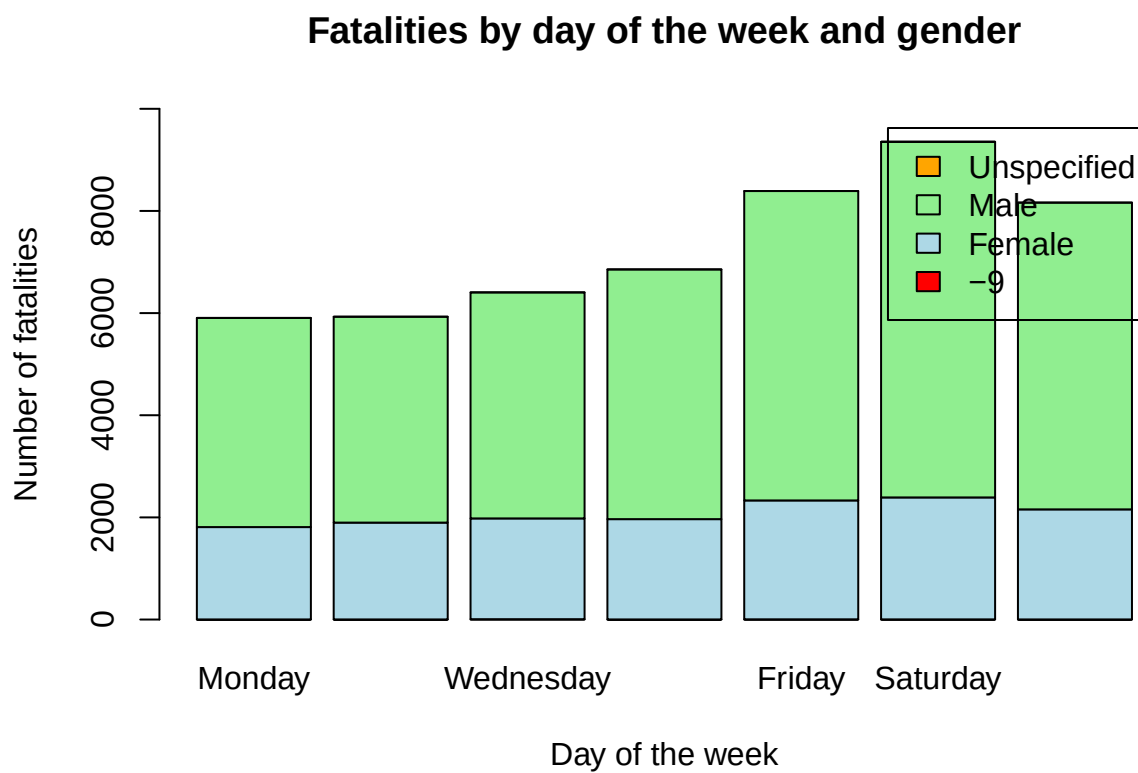


2.1.2 Double bar plot (frequency of 2 categorical variables) [Stacked or side-by-side]

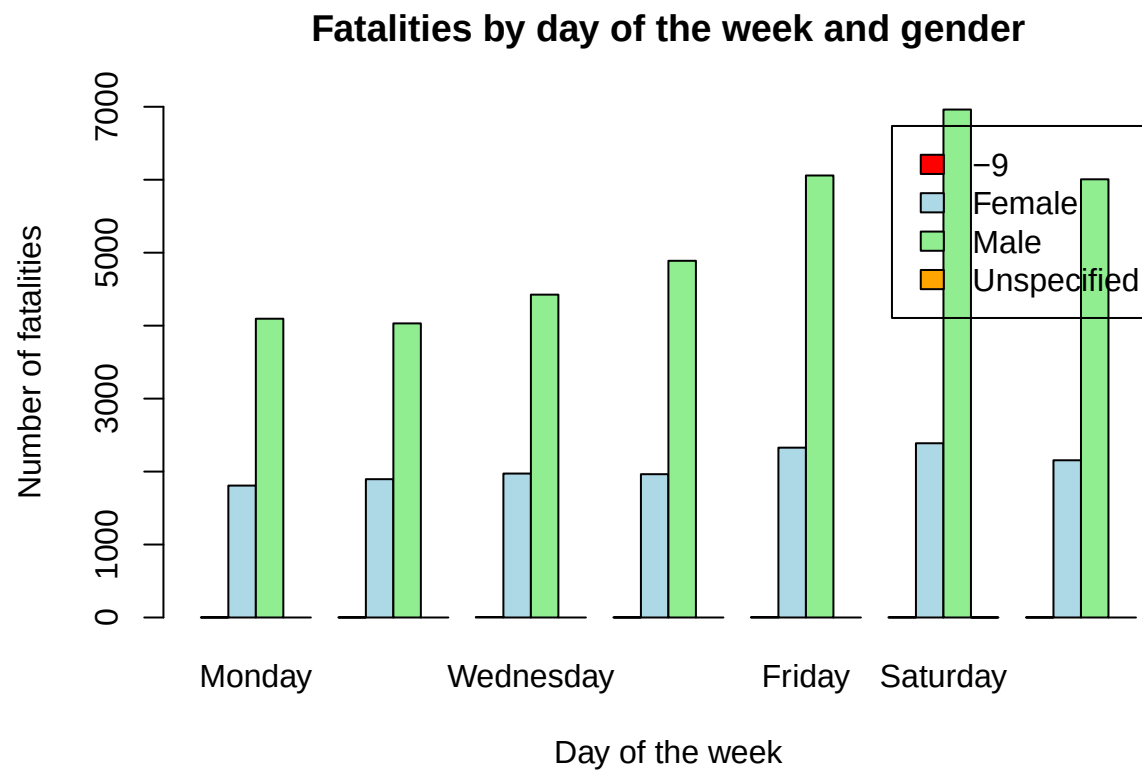
Similar to a single bar plot, but now we are comparing against 2 categorical variables.

```
gender <- data$Gender  
data1 <- table(gender, dayweek) # in the order that you want the grouping to follow
```

```
# Stacked bar plot  
barplot(data1,  
  main = "Fatalities by day of the week and gender",  
  xlab = "Day of the week",  
  ylab = "Number of fatalities",  
  ylim = c(0, 10000),  
  col = c("red", "lightblue", "lightgreen", "orange"),  
  legend = rownames(data1),  
  beside = FALSE)
```

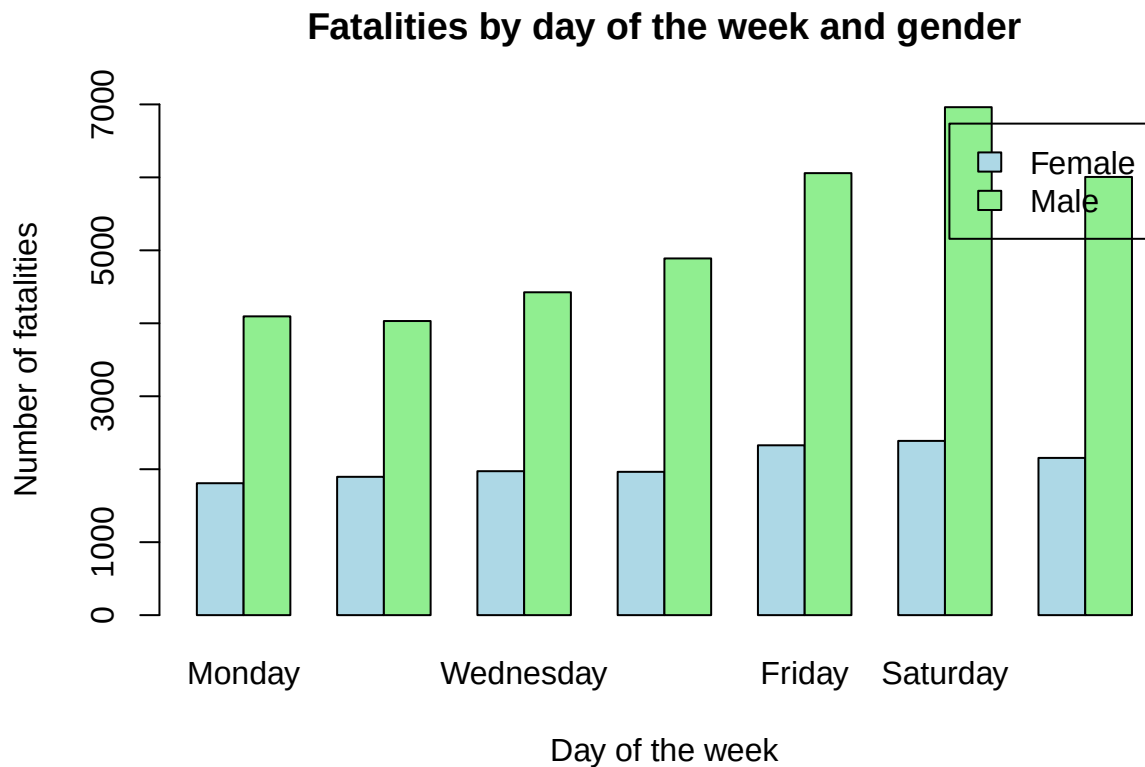


```
# Side-by-side bar plot
barplot(data1,
        main = "Fatalities by day of the week and gender",
        xlab = "Day of the week",
        ylab = "Number of fatalities",
        ylim = c(0, 7000),
        col = c("red", "lightblue", "lightgreen", "orange"),
        legend = rownames(data1),
        beside = TRUE)
```




```
# FILTERING OUT -9 AND UNSPECIFIED
data1_filtered <- data1[-c(1, 4), ] # we don't want rows 1 and 4, but we want all columns

# Side-by-side bar plot but filtered
barplot(data1_filtered,
        main = "Fatalities by day of the week and gender",
        xlab = "Day of the week",
        ylab = "Number of fatalities",
        ylim = c(0, 7000),
        col = c("lightblue", "lightgreen"),
        legend = rownames(data1_filtered),
        beside = TRUE)
```



```
speedlimit <- data$Speed.Limit
table(speedlimit)
```

```
## speedlimit
##      -9      <40      10      100      110      130
##    1381       6      17    17653    5796     106
##      15      20      25       30       40       5
##       1      26       2       13      321       3
##      50      60      70       75       80       90
##    2685   13371   2424      254    5843    1097
## Unspecified
##       2
```

It would make sense to represent speed limit as a categorical variable. We can also introduce a new category of <40 and merge speeds of 5 up to 30 as they are relatively low frequency.

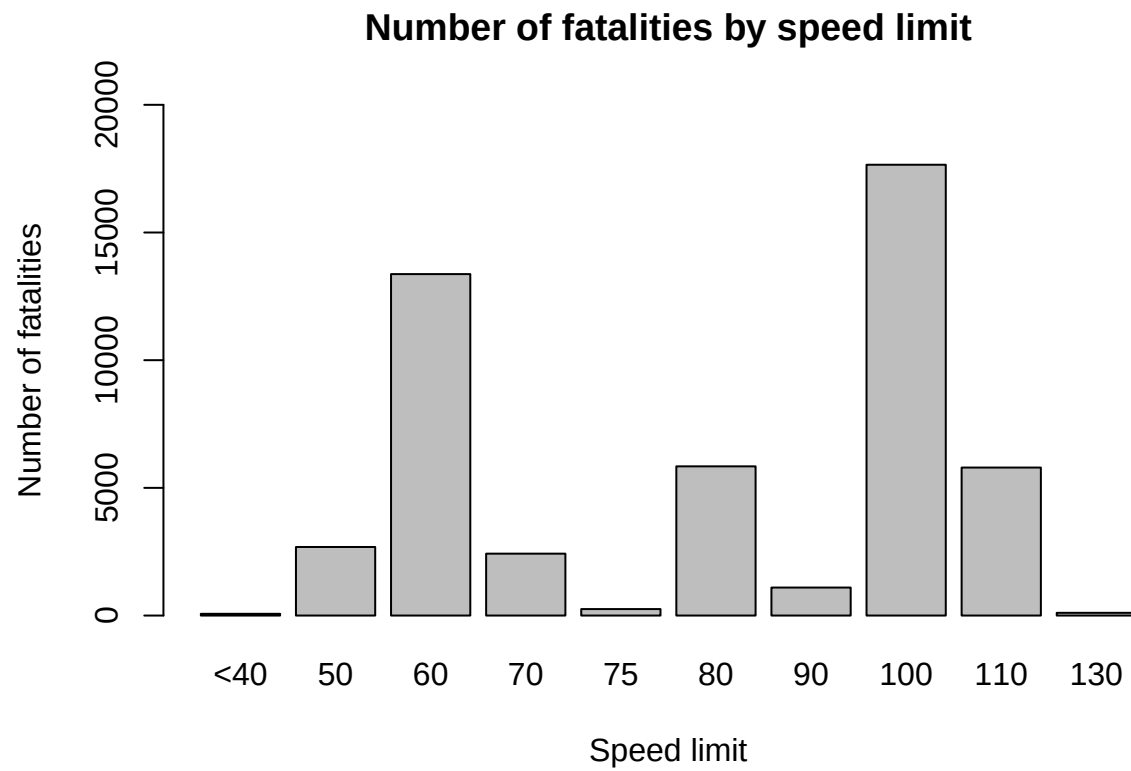
```
speedlimit[speedlimit == "5"] = "<40"
speedlimit[speedlimit == "10"] = "<40"
speedlimit[speedlimit == "15"] = "<40"
speedlimit[speedlimit == "20"] = "<40"
speedlimit[speedlimit == "25"] = "<40"
speedlimit[speedlimit == "30"] = "<40"
speedlimit[speedlimit == "-9"] = NA

# converting to a factor
speedlimit = factor(speedlimit,
                    levels = c("<40", "50", "60", "70", "75", "80", "90",
                              "100", "110", "130"))

table(speedlimit)
```

```
## speedlimit
##   <40   50   60   70   75   80   90  100  110  130
##    68 2685 13371 2424  254 5843 1097 17653 5796  106
```

```
# single bar plot
barplot(table(speedlimit),
        main = "Number of fatalities by speed limit",
        xlab = "Speed limit",
        ylab = "Number of fatalities",
        ylim = c(0, 20000))
```



```
# double bar plot
data2 = table(gender, speedlimit)
data2_filtered = data2[-c(1, 4), ]
barplot(data2_filtered,
        main = "Number of road fatalities by speed limit and gender",
        xlab = "Speed limit (km/h)",
        ylab = "Number of fatalities",
        col = c("lightblue", "lightgreen"),
        legend = rownames(data2_filtered),
        beside = TRUE)
```



2.2 Adjusting axis labels on a chart

There are three new arguments we provide to the charting function here: `las`, `cex.axis`, and `cex.names`.

2.2.1 `las`

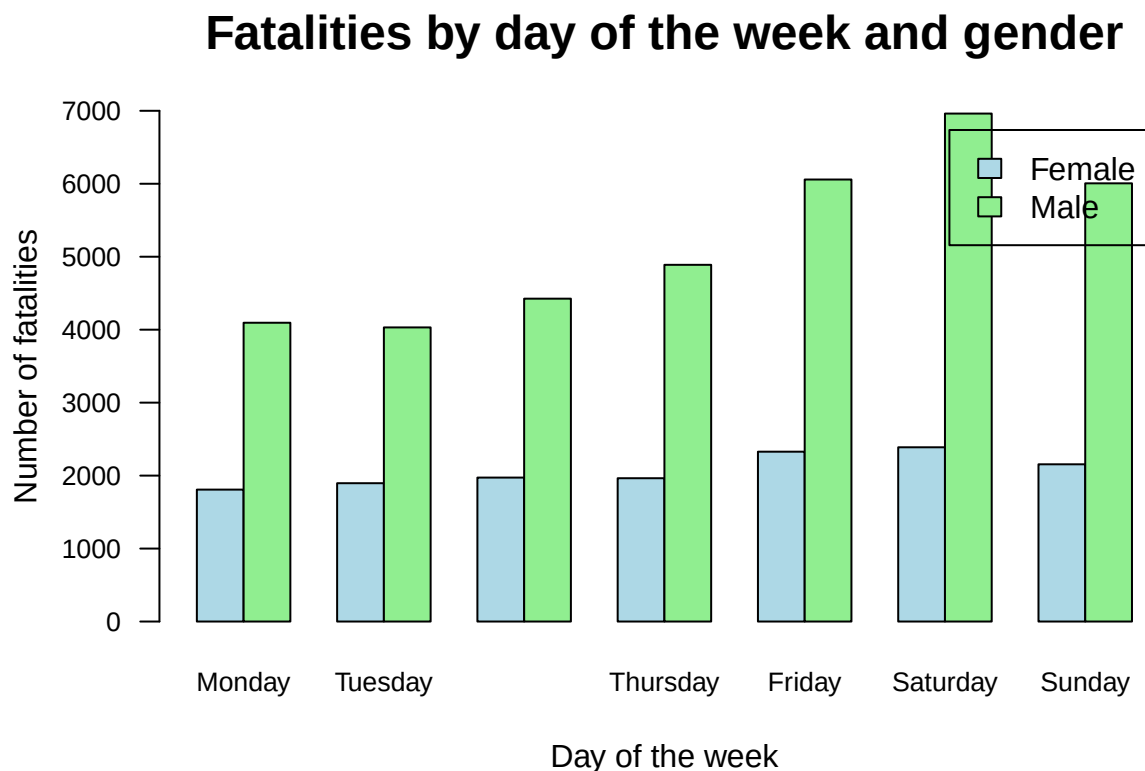
- `las = 0` : both labels are parallel to their axes
- `las = 1` : y-labels are perpendicular, x-labels are parallel
- `las = 2` : both labels are perpendicular to their axes
- `las = 3` : y-labels are parallel, x-labels are perpendicular

2.2.2 `cex`

`cex` is a function which controls the scaling of text in a graph.

- `cex.axis` controls the y-axis scaling
- `cex.names` controls the x-axis scaling
- `cex.main` controls the title scaling

```
# Side-by-side bar plot but prettier
barplot(data1_filtered,
  main = "Fatalities by day of the week and gender",
  xlab = "Day of the week",
  ylab = "Number of fatalities", ylim = c(0, 7000),
  col = c("lightblue", "lightgreen"),
  legend = rownames(data1_filtered),
  beside = TRUE,
  las = 1,
  cex.axis = 0.80, cex.names = 0.80, cex.main = 1.5)
```



```
qnorm(.9)
```

```
## [1] 1.281552
```

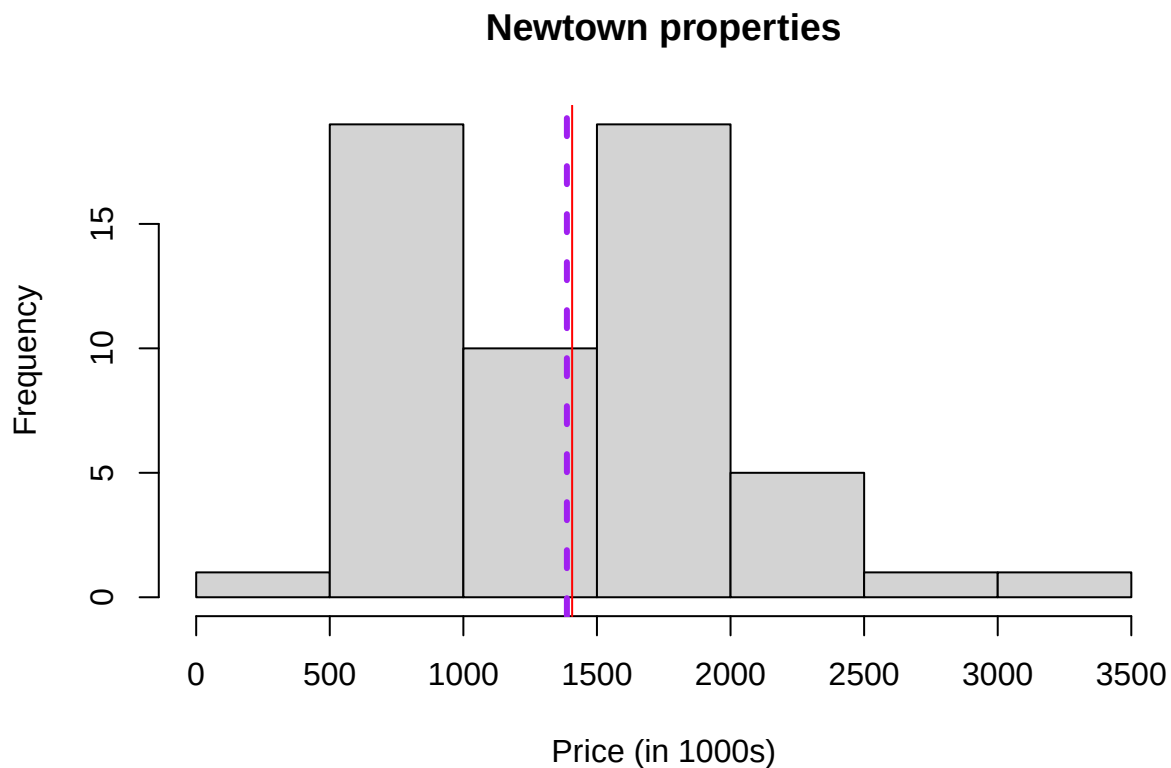
2.2.3 Adding lines and points to a graph

We can use the `abline` function to plot lines onto our charts. The line type (`lty`) can be 0 (blank), 1 (solid), 2 (dashed), 3 (dotted) and so on. Some use cases are:

- `abline(a = 1, b = 2, col = 'blue', lty = 2)` : plots a blue dashed line with intercept = 1 and slope = 2
- `abline(h = 4, col = 'green', lty = 1)` : plots a horizontal green solid line at $y = 4$
- `abline(v = mean(1, 2, 3, 4, 5))` : plots a vertical black solid line at $x = 3$

You can also use `lwd` (line width) to control the thickness of the lines.

```
data_houses <- read.csv("WEEK 3/NewtownJune2017.csv")
hist(data_houses$Sold,
     main = "Newtown properties",
     xlab = "Price (in 1000s)")
abline(v = mean(data_houses$Sold), col = "red")
abline(v = median(data_houses$Sold), col = "purple", lty = 2, lwd = 3)
```

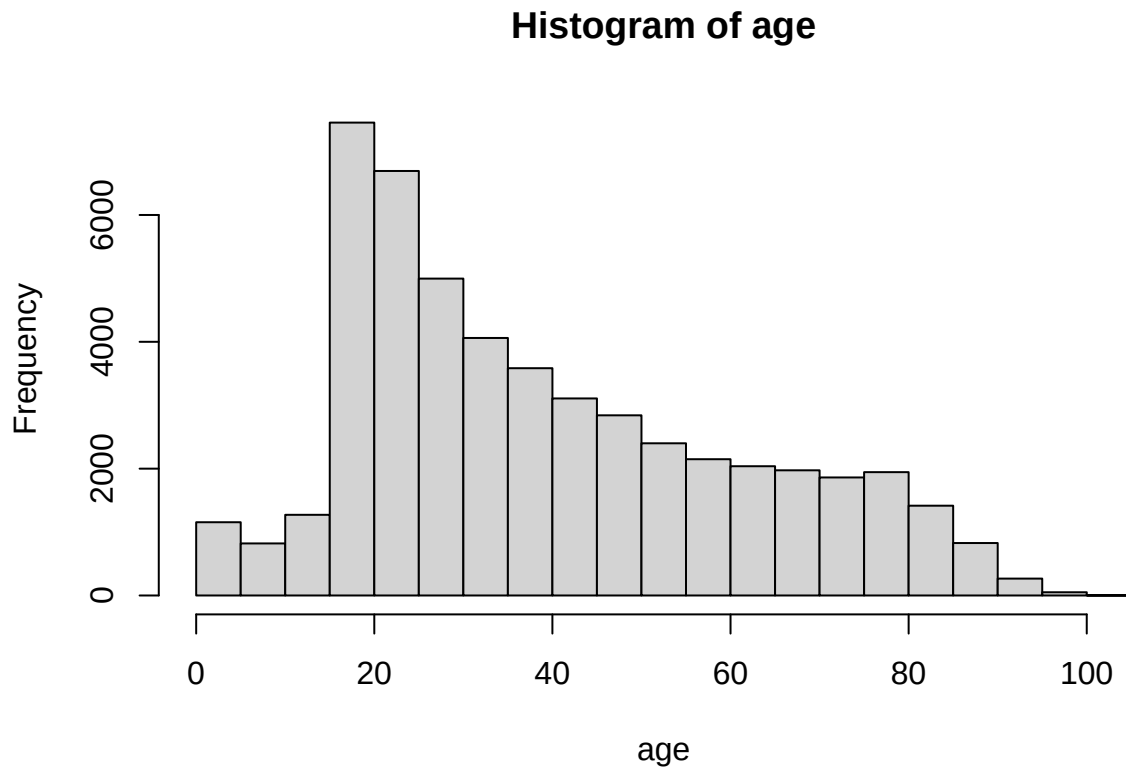


2.3 Quantitative data

2.3.1 Histogram (frequency of 1 quantitative variable)

```
data$Age[data$Age == -9] <- NA # cleaning
```

```
age <- data$Age  
hist(age)
```

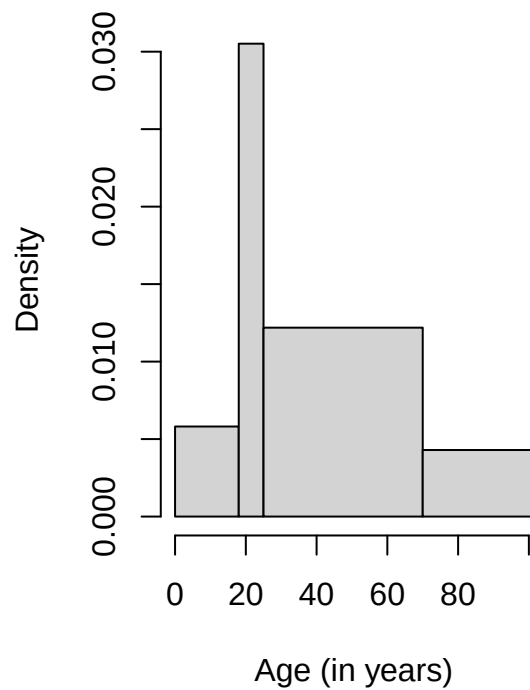


Note that this histogram is produced on the frequency scale. We prefer to use the density scale.

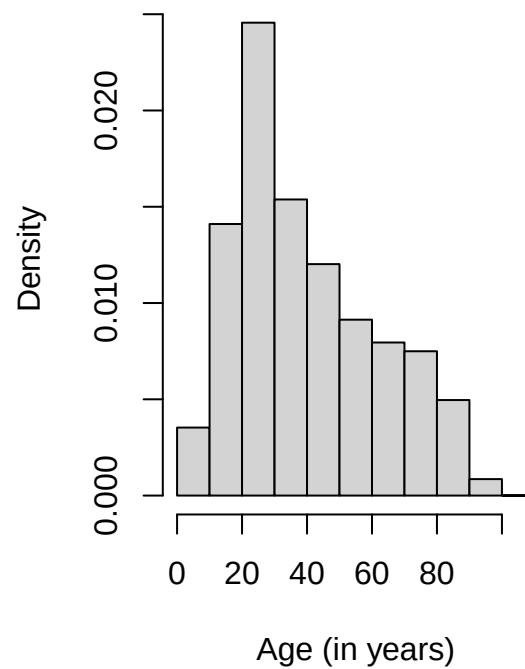

```
# choosing specific class intervals
par(mfrow = c(1, 2))
hist(age,
      br = c(0, 18, 25, 70, 101), # custom class intervals
      freq = FALSE, # produces the histogram on the density scale
      right = F, # makes the intervals right-open
      xlab = "Age (in years)",
      ylab = "Density")

hist(age,
      br = 10,
      freq = FALSE, # produces the histogram on the density scale
      right = F, # makes the intervals right-open
      xlab = "Age (in years)",
      ylab = "Density")
```

Histogram of age



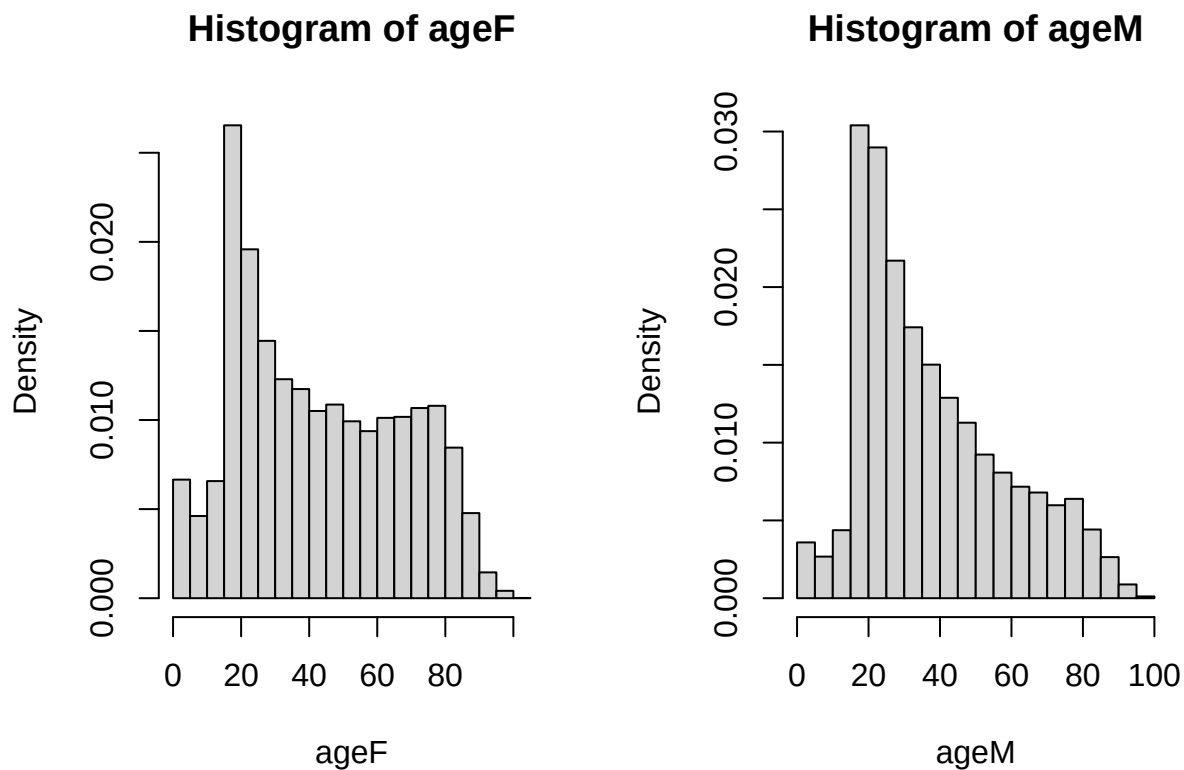
Histogram of age



2.3.2 Multiple histograms (frequency of 1 quantitative variable split by a categorical variable)

```
ageF <- data$Age[data$Gender == "Female"] # selecting just the female ages
ageM <- data$Age[data$Gender == "Male"]

par(mfrow = c(1, 2)) # outputting in 1 row and 2 columns
hist(ageF, freq = F)
hist(ageM, freq = F)
```



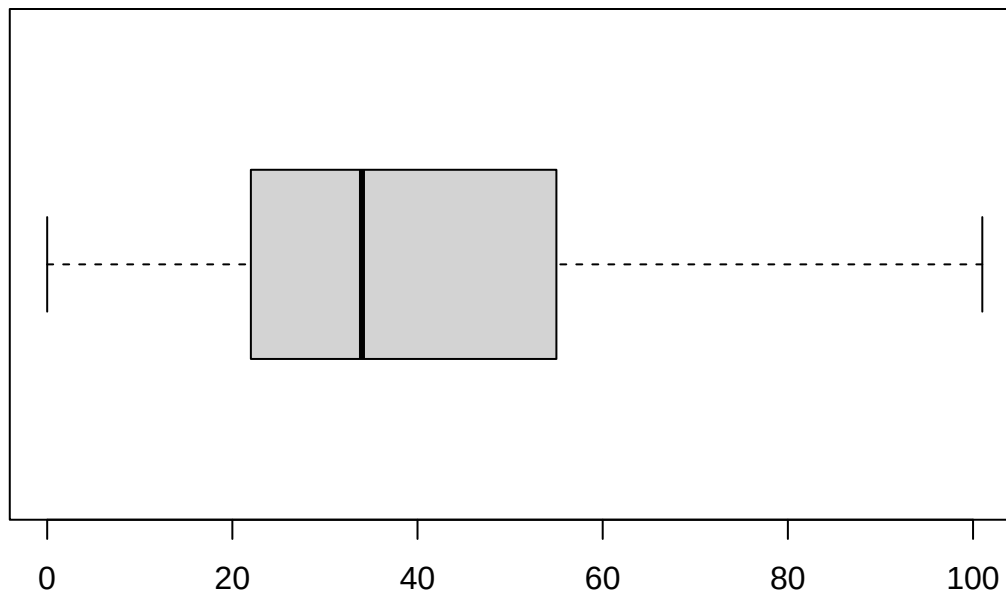
2.3.3 Single boxplot (numerical summary of a quantitative variable)

```
summary(age)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's  
##      0.0   22.0   34.0   39.6   55.0   101.0      84
```

```
boxplot(age,  
         main = "Boxplot of ages involved in fatalities",  
         horizontal = TRUE)
```

Boxplot of ages involved in fatalities



2.3.4 Multiple boxplots (numerical summary of a quantitative variable split up by a qualitative variable)

A comparative boxplot splits up a quantitative variable by a qualitative variable.

```
summary(age[gender == "Female"])
```

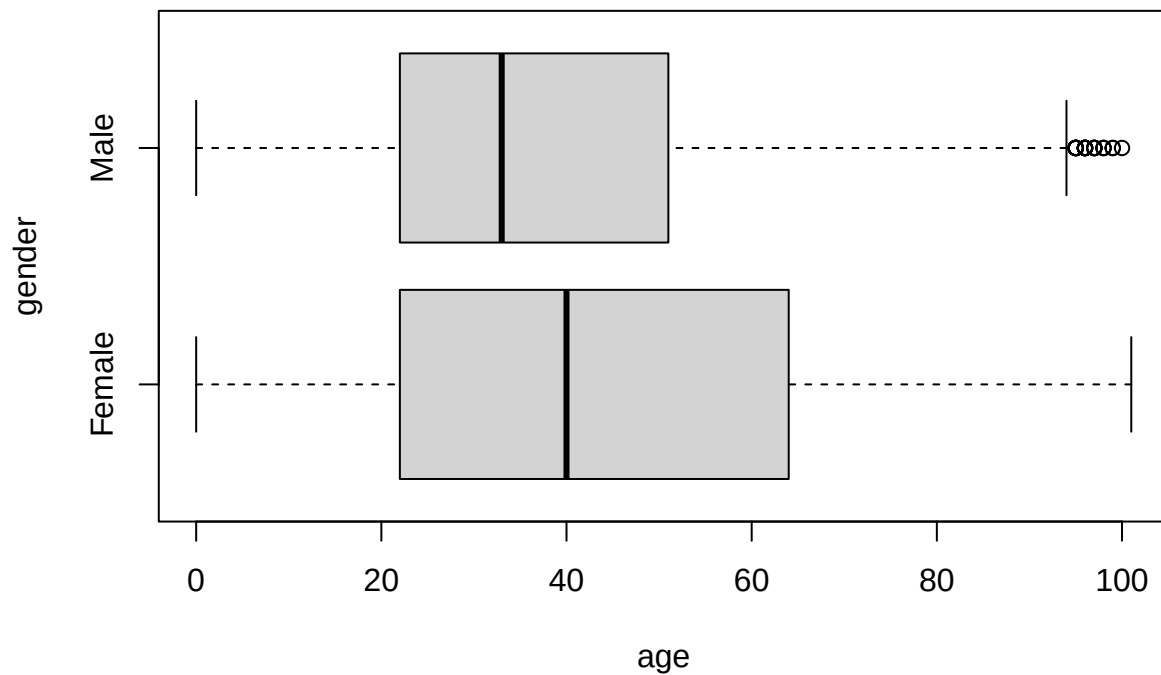
##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
##	0.0	22.0	40.0	43.1	64.0	101.0	29

```
summary(age[gender == "Male"])
```

##	Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
##	0.00	22.00	33.00	38.23	51.00	100.00	52

```
gender <- factor(gender, levels = c("Female", "Male"))
```

```
boxplot(age ~ gender, horizontal = TRUE) # plot gender split by age
```



2.3.5 Scatter plots (compare two quantitative variables)

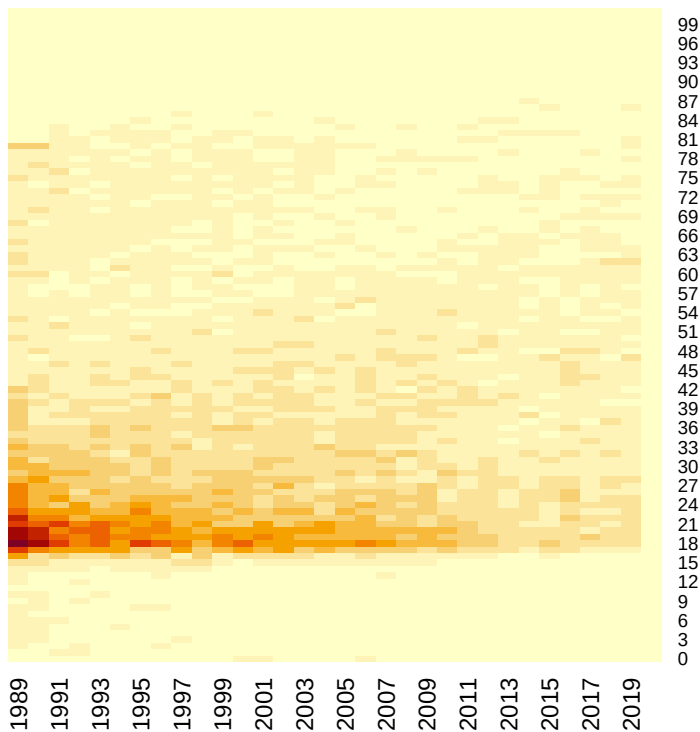
```
# plot()
```

2.3.6 Heatmaps

A heatmap is especially useful when a contingency table is not practical due to too many different values. We can also scale (centered and scaled in a specific direction), the heatmap by column — results in something like an Age histogram for every year.

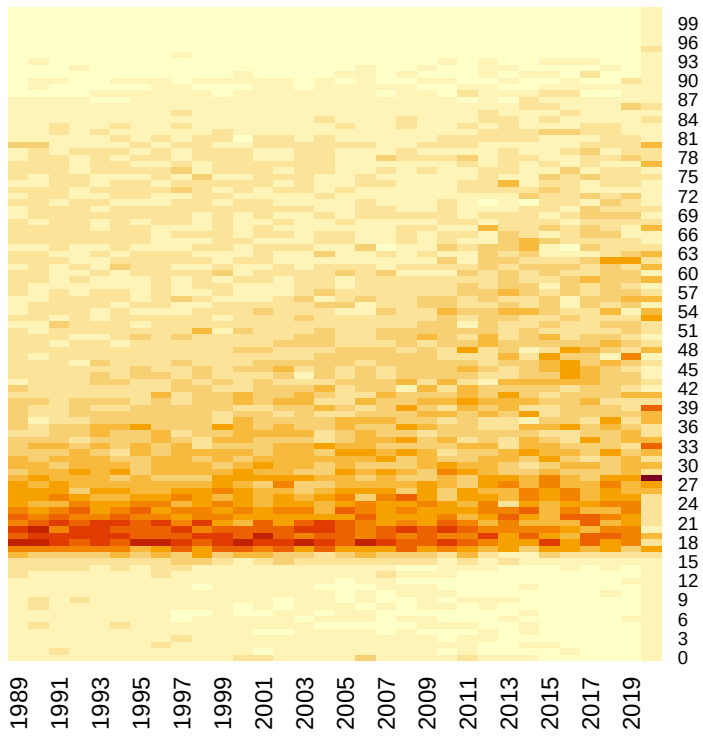
```
par(mfrow = c(1, 2))
heatmap(table(age, year),
        Rowv = NA,
        Colv = NA,
        scale = "none",
        main = "Heatmap of frequencies")
```

Heatmap of frequencies



```
# scaled by column
heatmap(table(age, year),
        Rowv = NA,
        Colv = NA,
        scale = "col",
        main = "Heatmap scaled by column")
```

Heatmap scaled by column



3 Numerical summaries: Centre

3.1 Sample mean

- The average of the data
- Involves all the data
- Serves as a balancing point

$$\bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$

In R we just use the `mean()` function.

```
mean(data_houses$Sold)
```

```
## [1] 1407.143
```

3.2 Sample median

The middle data point when the observations are ordered from smallest to largest.

For odd n :

$$\tilde{x} = x_{(\frac{n+1}{2})}$$

For even n :

$$\tilde{x} = \frac{x_{(\frac{n}{2})} + x_{(\frac{n}{2}+1)}}{2}$$

In R we use the `median()` function.

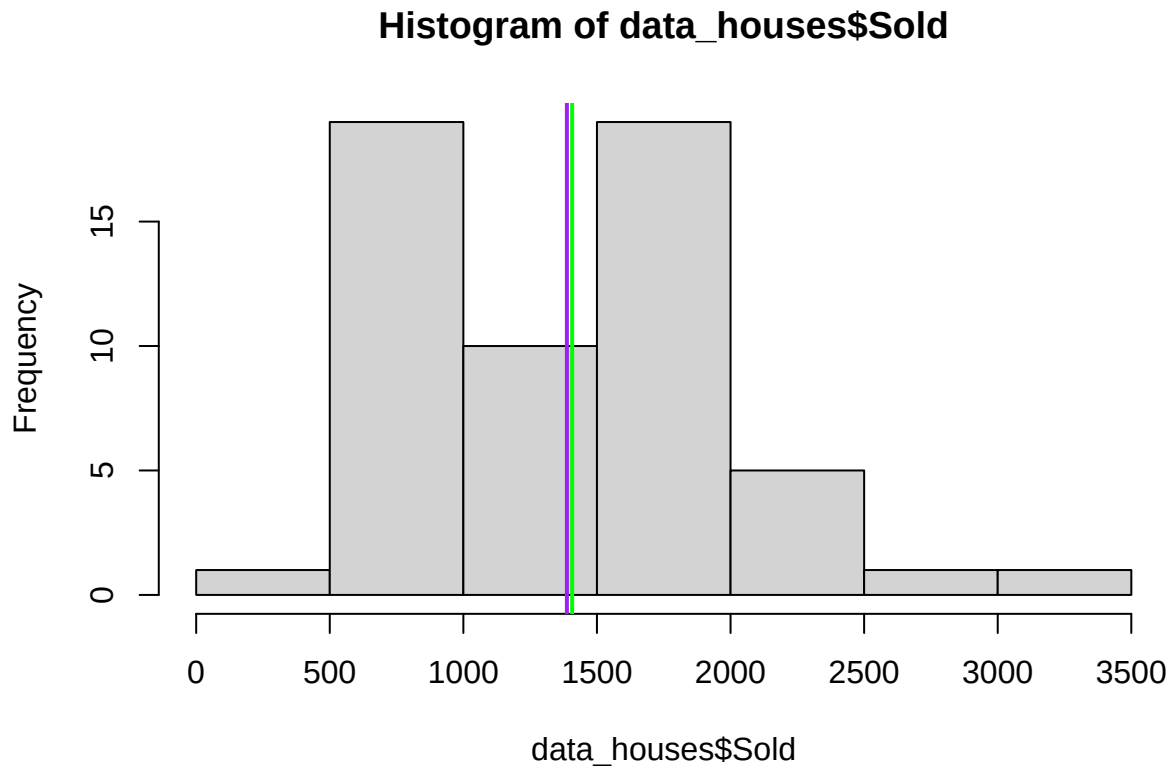
```
median(data_houses$Sold)
```

```
## [1] 1387.5
```

```
hist(data_houses$Sold)
```

```
abline(v = mean(data_houses$Sold), col = "green", lwd = 2)
```

```
abline(v = median(data_houses$Sold), col = "purple", lwd = 2)
```



The sample median is said to be robust and is a good summary for skewed data as it is not affected by outliers.

4 Numerical summaries: Spread

4.1 Sample SD

$$SD_{\text{sample}} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1}}$$

In R, the `sd` function **always** calculates the **sample standard deviation**.

You can convert from sample SD to population SD by a conversion factor:

$$SD_{\text{pop}} = SD_{\text{sample}} \times \sqrt{\frac{n - 1}{n}}$$

4.2 Population SD

$$SD_{\text{pop}} = \sqrt{\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n}}$$

In R, calculating the population SD requires the `multicon` package.

```
library(multicon)
```

```
## Loading required package: psych
## Loading required package: abind
## Loading required package: foreach
```



```
popsd(data_houses$Sold)
```

```
## [1] 593.7166
```

4.3 Z-Score

Measures how many SD's a point is above or below the mean.

$$z = \frac{x - \bar{x}}{SD}$$

4.4 IQR

Range of the middle 50% of the data.

$$IQR = Q_3 - Q_1$$

4.5 Lower and upper thresholds

$$LT = Q_1 - 1.5 \cdot IQR$$

$$UT = Q_3 + 1.5 \cdot IQR$$

4.6 Coefficient of variation

$$CV = \frac{SD}{\bar{x}}$$

5 General Normal Curve

Suppose the heights of Australian women follow a normal distribution with mean 161.8cm and SD 7.7cm. What proportion of women will have height less than 170cm?

We can use the `pnorm(x, mean, SD, lower.tail = TRUE/FALSE)` function to evaluate this.

```
pnorm(170, 161.8, 7.7, lower.tail = TRUE)
```

```
## [1] 0.8565473
```

6 Scatter plots and correlation

A scatter plot can be summarised by 5 numerical summaries:

- sample mean and sample SD of X ($\bar{x}$, SD_x)
- sample mean and sample SD of Y ($\bar{y}$, SD_y)
- correlation coefficient (r)

6.1 Correlation coefficient

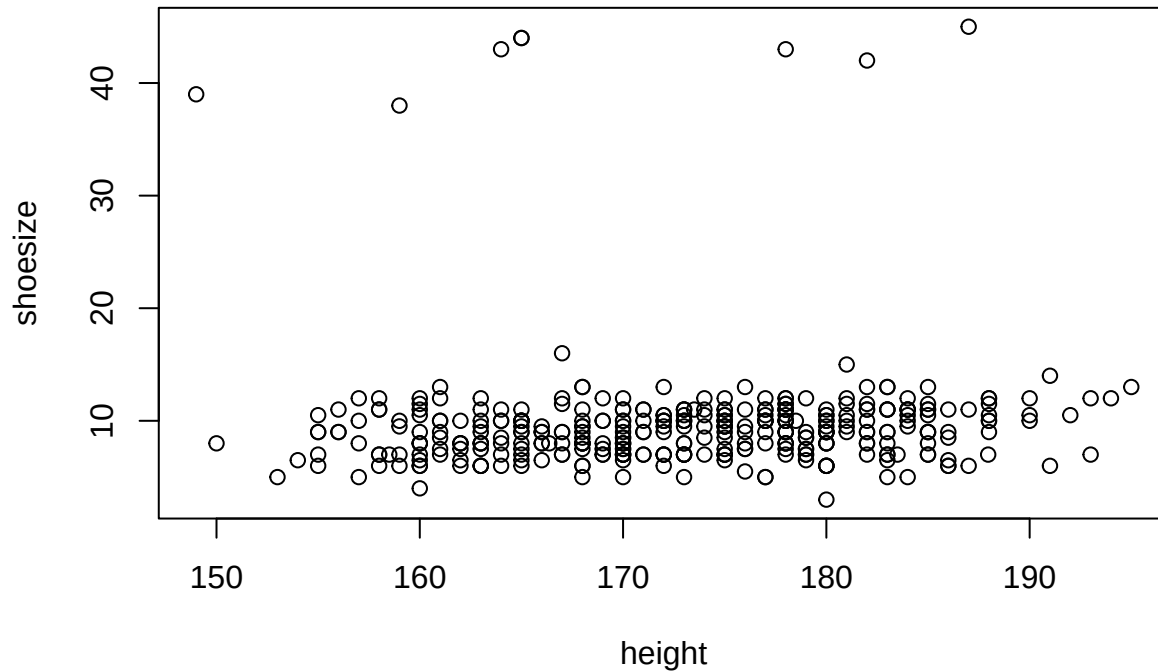
Measures the clustering of the data around a line. It is the mean of the product of the variables in standard units.

$$r = \frac{1}{n} \sum_{i=1}^n \frac{x_i - \bar{x}}{SD_x} \frac{y_i - \bar{y}}{SD_y}$$

```
data_class <- read.csv("WEEK 5/LAB/math1005.csv")
```

```
# data cleaning (removing na values)
height <- na.omit(data_class$Height)[1:358]
shoesize <- na.omit(data_class$ShoeSize)[1:358]
```

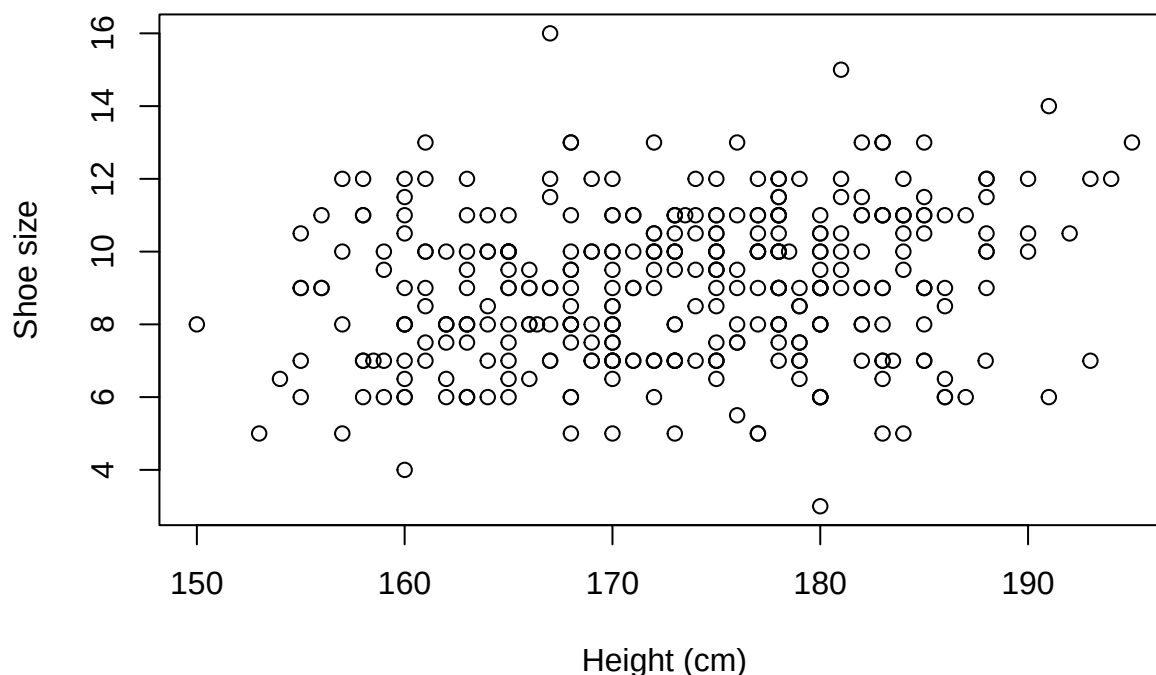
```
plot(height, shoesize)
```



```
# further data cleaning
# (removing any entries where the shoe size is greater than 20)
height <- height[shoesize <= 20]
shoesize <- shoesize[shoesize <= 20]
```

```
plot(height, shoesize,
      main = "Scatter plot of shoe sizes versus height",
      xlab = "Height (cm)",
      ylab = "Shoe size")
```

Scatter plot of shoe sizes versus height



```
library(multicon)
SU_x = (height - mean(height))/popsd(height) # converting to standard units
SU_y = (shoesize - mean(shoesize))/popsd(shoesize)
mean(SU_x * SU_y) # mean of the product of the variables in standard units
```

```
## [1] 0.1993456
```

In R, we can use the `cor(x, y)` function to calculate the correlation coefficient.

```
cor(SU_x, SU_y)
```

```
## [1] 0.1993456
```

6.2 Regression lines

6.2.1 SD line (bad descriptor)

The SD line connects the points of averages $(\bar{x}, \bar{y})$ to $(\bar{x} + SD_x, \bar{y} + SD_y)$. As it does not use the correlation coefficient, it is insensitive to the amount of clustering around the line. It generally under- and over-estimates at the extremes.

6.2.2 The regression line

To completely describe the scatter plot, we need to use all 5 numerical summaries: $\bar{x}$, $\bar{y}$, SD_x , SD_y and r .

The **regression line** connects $(\bar{x}, \bar{y})$ to $(\bar{x} + SD_x, \bar{y} + rSD_y)$. The r term in the second point's y -coordinate controls the steepness and the sign of the gradient. It improves the fit of the line at the extremes. Hence, the slope of the regression line is

$$m = \frac{(\bar{y} + rSD_y) - \bar{y}}{(\bar{x} + SD_x) - \bar{x}} = r \cdot \frac{SD_y}{SD_x}$$

The y-intercept of the regression line is

$$b = \bar{y} - m\bar{x}$$

In R, we use the `lm(y ~ x)` function (linear model) to find the regression line. The input is $y \sim x$ as in “ y variable regressed onto x variable”.

```
linear_model <- lm(shoesize ~ height)
print(linear_model)
```

```
##
## Call:
## lm(formula = shoesize ~ height)
##
## Coefficients:
## (Intercept)      height
##      1.33390      0.04451
```

The `lm` function has many callable parameters such as `$coefficients` which returns a named vector of coefficients where [1] is the intercept and [2] is the slope.

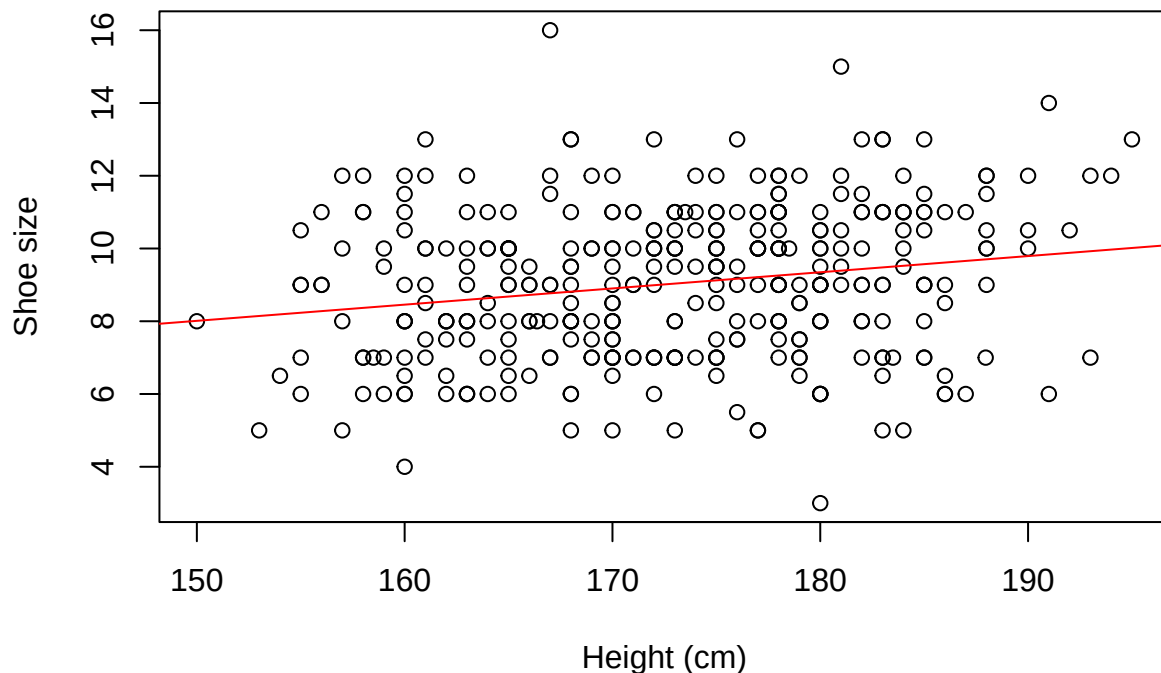
```
linear_model$coefficients
```

```
## (Intercept)      height
##      1.333899      0.044514
```

So the regression line for this linear model is $y = 1.33 + 0.0445x$. To plot this regression line onto the scatter plot we can use `abline()`.

```
plot(height, shoesize,
      main = "Scatter plot of shoe sizes versus height",
      xlab = "Height (cm)",
      ylab = "Shoe size")
abline(lm(shoesize ~ height), col = "red")
```

Scatter plot of shoe sizes versus height



The best method for making predictions is the regression line as it includes all the data. You can use the regression line to predict y from x or x from y but you **must refit the data**: you cannot simply rearrange

the formula.

6.3 Residuals

A residual is the vertical distance of a point above or below the regression line. It represents the error between the actual value and the prediction.

$$e_i = y_i - \hat{y}_i$$

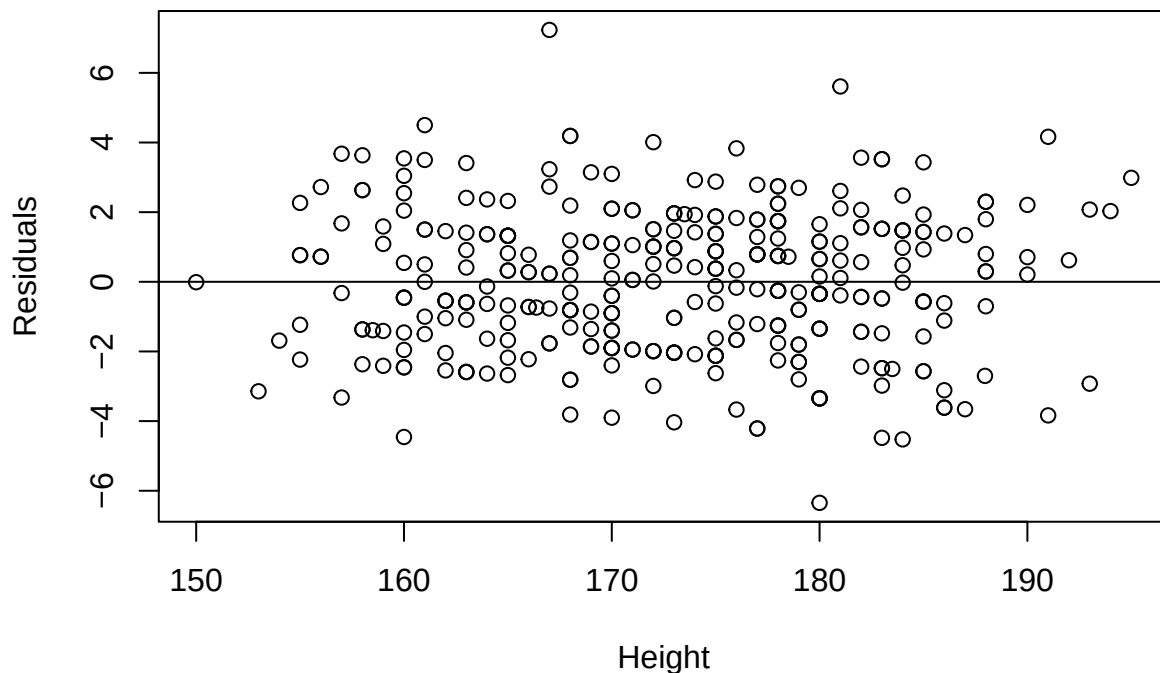
where y_i is the actual value and $\hat{y}_i$ is the prediction.

6.3.1 Residual plot

- A residual plot graphs the residuals vs x .
- The residual plot is a diagnostic plot to check the appropriateness of a linear model.
- If the linear fit is appropriate for the data, it should show no pattern (random about $y = 0$).

In R, the residuals are built into the linear model.

```
plot(height, linear_model$residuals, ylab = "Residuals", xlab = "Height")
abline(h = 0)
```



- If there is the same (equal) variance around the y axis, the data displays *homoscedasticity* and the data is appropriate to be fit using a linear model. The regression line can be used for predictions.
- If not, the data displays *heteroscedasticity* and the data should not be fit using a linear model.

7 Chance

7.1 Simulating chance

Write a simulation for a game where you toss a die 4 times and you win if you get at least one 1.

```
gameA <- function() {
  rolls <- sample(1:6, size = 4, replace = TRUE)
  condition <- sum(rolls == 1) > 0
  return(condition)
}
```

```
}  
  
simsA <- replicate(1e+05, gameA()) # results in a table of T/F values if won/lost  
probability <- sum(simsA)/length(simsA) # calculates the probability of winning  
probability  
  
## [1] 0.51702
```